L'Ecuyer-CMRG

Jesús F García Gavilán

Reproducibilidad en computación paralela con L'Ecuyer-CMRG en R: propuesta de protocolo

Objetivo: Garantizar que los análisis estadísticos, modelos de machine learning y simulaciones ejecutados en paralelo reproduzcan los resultados en cada ejecución, manteniendo el rigor metodológico (especialmente importante en bases de datos de modelos para la salud)

Requisitos previos: Tener instalado R. No se requieren paquetes externos. Para este protocolo utilizaremos el paquete `parallel` incluido en R base

Paso 1: Configurar el motor de números aleatorios

Antes de iniciar cualquier proceso, el entorno global de R debe configurarse para usar el algoritmo adecuado para el paralelismo. Para ello se debe:

1. Carga la librería `parallel`
2. Cambia el motor de generación de números aleatorios al algoritmo de *L'Ecuyer-CMRG* mediante `RNGkind`
3. Fija una semilla global para el entorno principal

Paso 2: Instanciar el clúster de procesamiento

Debes preparar los núcleos de tu procesador para trabajar simultáneamente sin saturar el sistema operativo

4. Calcula el número de núcleos disponibles en tu máquina restando al menos uno (`detectCores() - 1`). Yo recomiendo liberar 2 o 3, sobre todo si quieres seguir trabajando en otras tareas
5. Crea el clúster de trabajo con la función `makeCluster`

Paso 3: Sincronizar las semillas en los nodos (El paso clave)

Para que cada hilo de trabajo produzca secuencias independientes y deterministas, debes enviar la semilla al clúster. Para ello, utiliza la función `clusterSetRNGStream` asignándole el clúster creado y una semilla específica (`iseed`)

Paso 4: Ejecutar y liberar recursos

Una vez preparado el entorno, lanza tu función paralela y cierra el clúster al terminar para liberar la memoria (importante!!)

6. Ejecuta tu función utilizando `parLapply` (o funciones equivalentes de la familia apply en paralelo)
7. Detén **siempre** el clúster con `stopCluster` al finalizar (el siempre es para acordarse de finalizar procesos)

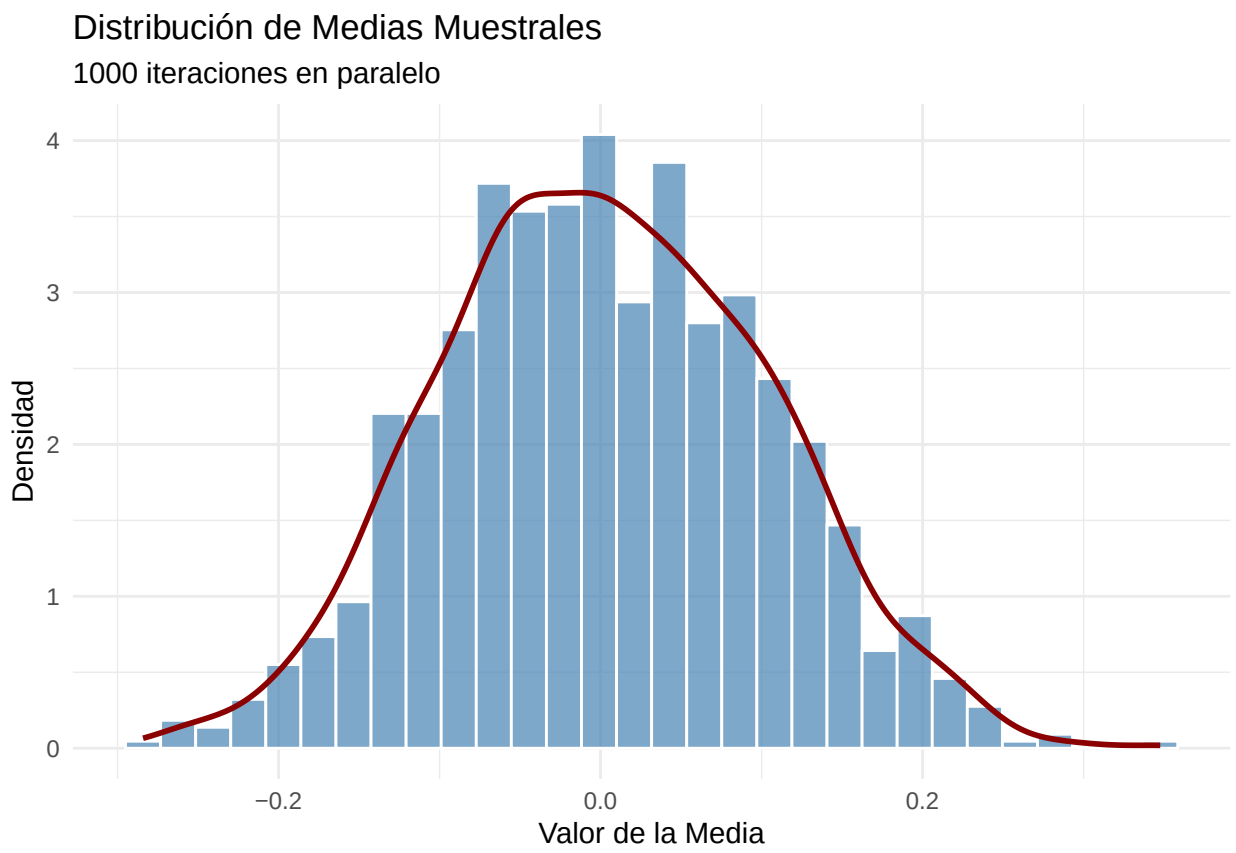
Código estándar de implementación

Abajo os dejo un código para copiar y adaptar en vuestros scripts:

```r
# 1. Cargar el paquete para computación paralela
library(parallel)
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```

```r
# 2. Configurar el algoritmo generador de números aleatorios
RNGkind("L'Ecuyer-CMRG")
set.seed(123)

# 3. Detectar núcleos y crear el clúster (dejando 1 núcleo libre para el SO)
num_cores = detectCores() - 1
cl = makeCluster(num_cores)

# 4. Distribuir secuencias de números independientes a cada núcleo
clusterSetRNGStream(cl, iseed = 123)

# 5. Ejecutar el análisis en paralelo
# (Ejemplo: Calcular la media de 100 números aleatorios normales, iterado 1000 veces)
resultados = parLapply(cl, 1:1000, function(x) mean(rnorm(100)))

# 6. Detener el clúster y liberar los recursos del procesador
stopCluster(cl)

# 7. Visualización de los resultados:
datos = data.frame(medias = unlist(resultados))

ggplot(datos, aes(x = medias)) +
  geom_histogram(aes(y = after_stat(density)), bins = 30,
                 fill = "steelblue", color = "white", alpha = 0.7) +
  geom_density(color = "darkred", linewidth = 1) +
  theme_minimal() +
  labs(title = "Distribución de Medias Muestrales",
       subtitle = "1000 iteraciones en paralelo",
       x = "Valor de la Media",
       y = "Densidad")
```



Nota Metodológica: La reproducibilidad computacional lograda con este esquema no es opcional al escalar análisis, es la base sobre la que se construye cualquier proyecto de ciencia de datos fiable aplicado a la salud pública o la biomedicina